

CoC 跑团平台 PRD (Product Requirements Document)

文档版本: v1.0

创建日期: 2026-02-05

最后更新: 2026-02-05

产品状态: 规划阶段

文档变更历史

版本	日期	作者	变更说明
v1.0	2026-02-05	-	初始版本, 基于原始需求文档整理

目录

- 产品概述
- 目标用户
- 核心价值
- 产品范围
- 功能需求
- 非功能需求
- 用户体验设计原则
- 数据模型
- 里程碑规划
- 成功指标

1. 产品概述

1.1 产品定义

CoC 跑团平台是一款基于大语言模型 (LLM) 的《克苏鲁的呼唤》(Call of Cthulhu 7th Edition) 在线跑团 (TRPG) 平台。系统通过 AI 扮演 Keeper (主持人/KP), 支持多名玩家通过自然语言进行角色扮演, 自动处理游戏机制 (检定、战斗、SAN值等), 并提供完整的长团记忆、规则检索和 Web 管理界面。

1.2 核心问题解决

当前在线跑团面临的痛点:

痛点	现状	我们的解决方案
KP 门槛高	需要 KP 熟读规则书、准备模组、主持流程，学习成本巨大	AI 自动处理规则，玩家只需自然语言表达意图
组团困难	需要协调时间、寻找合适的 KP 和玩家	云端服务，随时发起/加入，AI 作为 24/7 可用的 KP
机制繁琐	检定、战斗、SAN 值计算需要大量人工操作	自动化机制处理，所有变更可审计可追溯
长团难续	线索、承诺、状态容易遗忘	结构化记忆系统，随时检索历史，断点续跑
规则争议	规则解释不一，容易产生分歧	规则书知识库检索 + 桌规裁定系统，可追溯

1.3 产品定位

- 类型：AI 驱动的垂直领域应用（TRPG 辅助平台）
- 细分领域：CoC 7e 规则系统在线跑团
- 差异化：真正的"自然语言 + 自动机制"闭环，而非简单的聊天机器人
- 目标：成为 CoC 跑团的标准在线平台

2. 目标用户

2.1 用户画像

主要用户画像

用户类型	占比	特征	核心需求	使用场景
TRPG 新手	40%	对跑团好奇但缺乏经验，不了解 CoC 规则	低门槛入坑，AI 引导机制，友好的拒绝和提示	第一次尝试跑团，学习规则
经验玩家	50%	熟悉 TRPG 基本概念，可能玩过 D&D 或其他系统	快速组局，自动化减少繁琐操作，可靠的长团记忆	日常跑团，短模组体验
KP/主持人	10%	有经验但时间和精力有限，需要辅助工具	模组管理，玩家管理，AI 作为辅助而非替代	准备模组，管理长团，复盘回顾

2.2 用户使用场景

场景 1：新手第一次体验

小明对跑团很感兴趣，但身边没有有经验的朋友带。他在搜索中发现了我们的平台，注册后选择了一个入门模组，用自然语言描述自己的行动，AI KP 引导他完成了第一个调查、检定和战斗，让他感受到跑团的乐趣。

场景 2：老玩家的快速对局

老张和朋友们想跑一个短模组，但没人有时间准备 KP。他们在平台上创建 Campaign，分配好角色卡，用自然语言推进剧情，系统自动处理所有检定和战斗，2 小时完成一个完整的模组体验。

场景 3：长团的中断与恢复

一个跑了一半的模组因为大家时间对不上暂停了两周。当玩家们回来时，通过 `/recap` 和 `/memory` 命令快速回忆起关键线索和承诺，继续推进剧情，没有任何信息丢失。

3. 核心价值

3.1 用户价值

价值维度	具体体现
降低门槛	不需要学习规则书，不需要经验丰富的 KP，自然语言即可游玩
节省时间	自动化机制处理，KP 无需手动计算，玩家无需等待
可审计性	所有机制变更可追溯，避免"黑箱操作"，规则透明
持续体验	长团记忆系统，支持中断后随时恢复，线索永不丢失
社交连接	云端组局，随时随地与朋友或陌生人一起跑团

3.2 平台价值（商业视角）

价值维度	具体体现
垂直领域垄断	CoC 是最受欢迎的 TRPG 系统之一，专注可建立壁垒
用户粘性	长团特性带来高留存，Campaign 可持续数月
内容生态	模组/角色卡 UGC 社区，形成网络效应
技术壁垒	AI + 机制 + 记忆系统的复杂集成，难以快速复制

4. 产品范围

4.1 功能边界（In Scope）

产品形态

本产品为纯 Web 应用，用户通过浏览器访问所有功能。不提供桌面客户端或移动 App。

v1.0 核心功能

模块	后端功能	前端界面	优先级
用户系统	注册/登录/鉴权 RBAC权限 (KP/Player) 资源归属管理	登录/注册页 个人中心 权限控制的路由	P0
核心玩法	自然语言交互 (TRPG-only 门禁) 检定系统 (属性/技能, 奖励/惩罚骰) 推骰/花幸运机制 战斗系统 (伤害/重伤/濒死/治疗) 追逐系统 (距离/压力/障碍) SAN 系统 (检定/扣除/疯狂/恢复)	游戏台 (聊天式交互界面) 实时状态面板 机制结果可视化 (骰子/伤害/SAN) 快捷操作按钮	P0
机制与规则	规则书知识库检索 桌规裁定与追溯 线索账本与 Leads 机制 可审计的事件日志	规则搜索界面 线索面板 事件查看器	P0
记忆系统	全文事件日志 结构化摘要 (检查点/场景/Session) 记忆检索与引用 断点续跑与状态恢复	复盘界面 记忆搜索框 时间线展示	P0
多人功能	多人加入与身份管理 聚光灯/队列系统 可见性控制 (公开/KP-only/私密) 权限与鉴权	多人席位显示 实时聊天 聚光灯指示器 私密消息提示	P0
内容管理	模组上传与场景包校验 角色卡创建/编辑/导入导出 Campaign 管理	脚本库界面 (列表/详情/上传) 角色卡编辑器 Campaign 管理界面 拖拽上传组件	P0
体验优化	防卡死与失败前进 友好的拒绝与引导 输出格式化与信息密度控制	加载动画 错误提示组件 响应式布局 (支持平板) 暗色主题 (可选)	P1

4.2 明确排除 (Out of Scope)

排除项	原因	未来可能
通用 AI 助手功能	聚焦 TRPG 垂直领域, 避免 scope 蔓延	✗ 不考虑
非 CoC 规则系统	v1 专注 CoC 7e 做深做透	✓ v2+ 可能支持 D&D/其他
实时语音/视频	技术复杂度高, 非核心价值	✓ 作为独立功能考虑
模组创作工具	v1 仅支持上传现成模组	✓ 作为创作者平台功能
移动端原生 App	Web 响应式设计已可满足移动需求, 原生App开发成本高	✓ 用户量达到规模后再评估
桌面客户端	Web 应用已足够, 跨平台无需额外开发	✗ 不考虑
区块链/NFT 元素	与产品定位不符	✗ 不考虑

4.3 技术架构

技术选型（已确定）

本产品采用以下技术栈：

前端：React + shadcn/ui

后端：Python + Agno 框架

前端架构

组件	技术选型	说明
框架	React 18+	组件化开发，Hooks 生态
UI 组件库	shadcn/ui	基于 Radix UI，高度可定制
状态管理	Zustand	轻量级，适合复杂游戏状态
实时通信	WebSocket (Socket.io)	多人实时交互
路由	React Router v6	客户端路由
表单	React Hook Form + Zod	类型安全的表单验证
富文本	Tiptap	角色卡/模组编辑
样式	Tailwind CSS	shadcn/ui 原生支持
构建工具	Vite	快速开发体验
类型检查	TypeScript	类型安全

前端目录结构（规划）：

```
src/
├── components/           # 通用组件
│   ├── ui/              # shadcn/ui 组件
│   ├── game/            # 游戏相关组件
│   │   ├── GameConsole.tsx
│   │   ├── StatePanel.tsx
│   │   ├── MessageBubble.tsx
│   │   └── DiceRoll.tsx
│   └── layout/           # 布局组件
├── pages/                # 页面组件
├── hooks/                # 自定义 Hooks
├── store/                # Zustand 状态管理
├── services/             # API 调用
├── types/                # TypeScript 类型
└── utils/                # 工具函数
```

后端架构

组件	技术选型	说明
框架	Agno (Python)	AI 应用框架, 内置 LLM 集成
AI 集成	Agno LLM 组件	支持 OpenAI/Claude/本地模型
API	FastAPI (Agno 内置)	RESTful + WebSocket
数据库	PostgreSQL (主库)	关系型数据存储
缓存	Redis	会话缓存、实时状态
向量检索	Agno Vector Store	记忆/规则检索 (支持 pgvector/Pinecone)
消息队列	Celery + Redis	异步任务处理
认证	JWT + OAuth2	用户认证与授权
部署	Docker	容器化部署

Agno 框架优势:

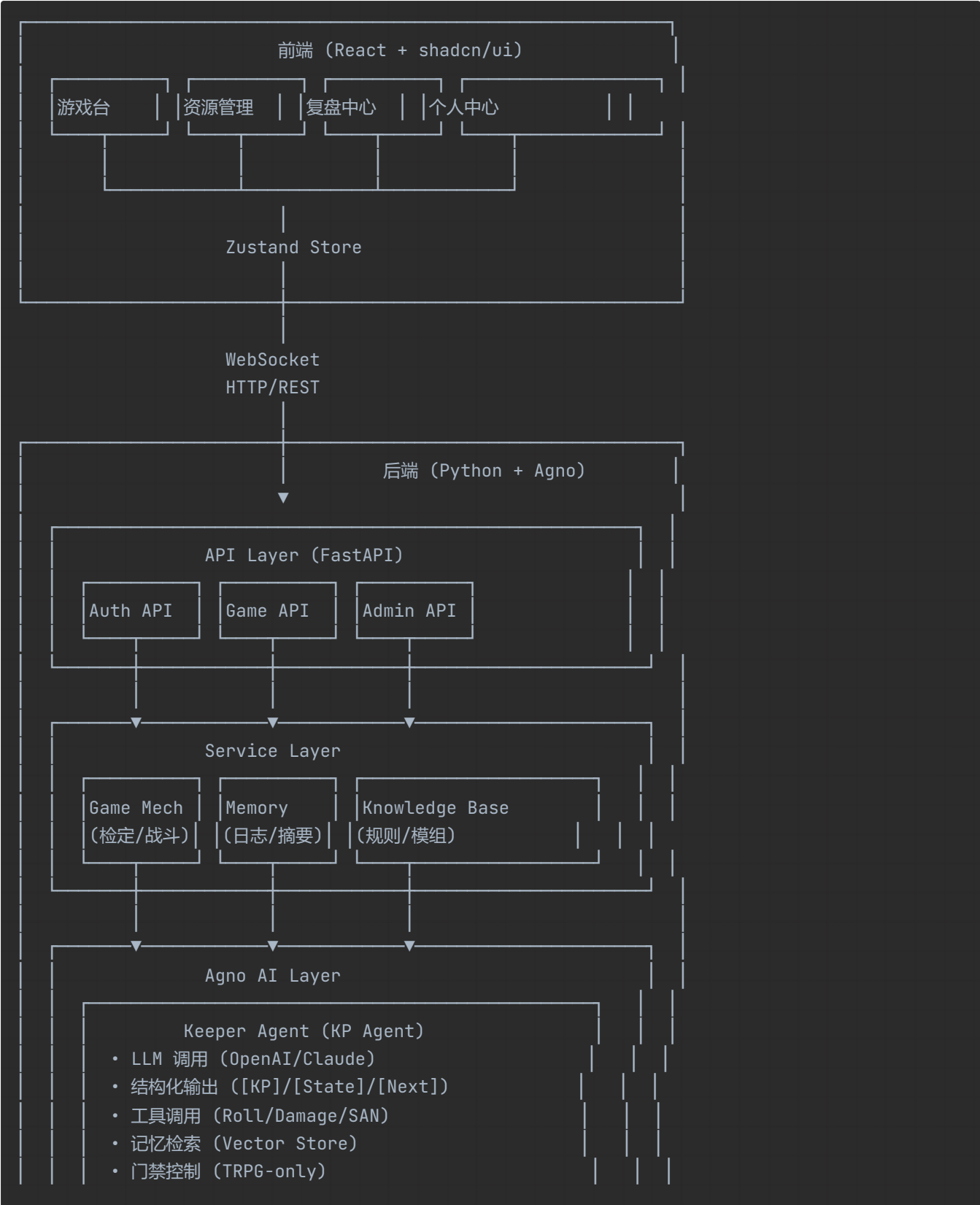
- 🤖 内置 LLM 集成 (支持多模型切换)
- 📦 开箱即用的向量存储与检索
- 🛠️ 原生支持工具调用 (Tool Calling)
- 🏗️ 结构化输出验证 (Pydantic)
- 🧑‍🔬 Agent 编排能力
- 🎯 专为 AI 应用设计

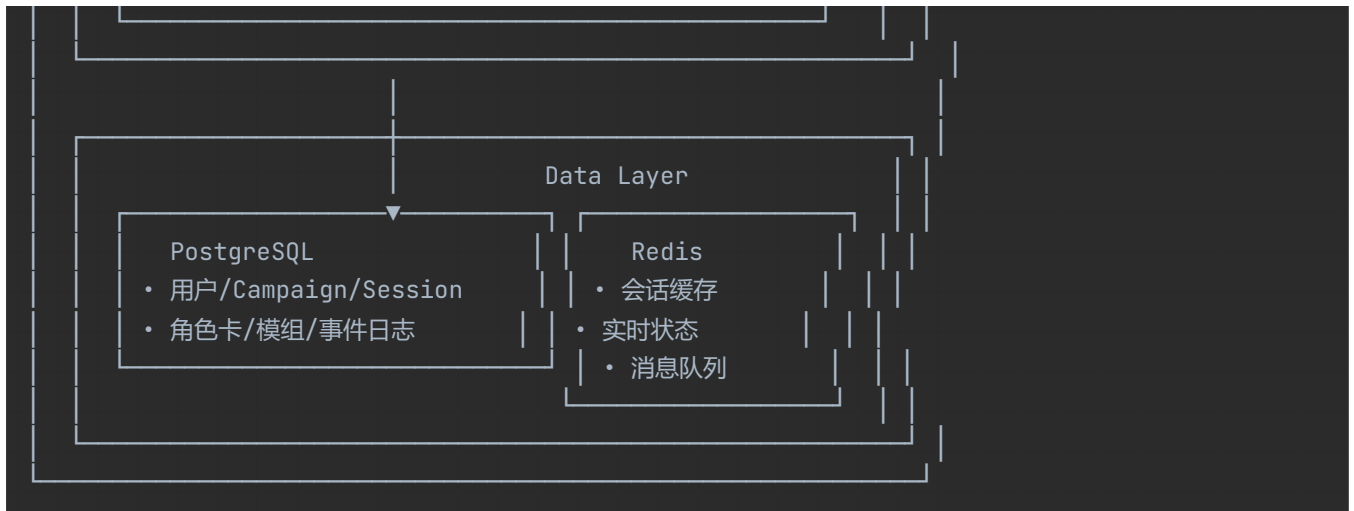
后端目录结构 (规划) :

```
backend/
├── app/
│   ├── api/                # API 路由
│   │   ├── auth.py
│   │   ├── campaigns.py
│   │   ├── sessions.py
│   │   └── websocket.py
│   ├── core/               # 核心配置
│   │   ├── config.py
│   │   ├── security.py
│   │   └── deps.py
│   ├── models/             # 数据库模型
│   ├── schemas/            # Pydantic 模型
│   ├── services/           # 业务逻辑
│   │   ├── game/          # 游戏机制
│   │   │   ├── dice.py
│   │   │   ├── combat.py
│   │   │   └── sanity.py
│   │   └── ai/             # AI 相关
│   │       ├── keeper.py  # KP Agent
│   │       ├── memory.py  # 记忆管理
│   │       └── kb.py       # 知识库
```

```
├── auth.py
├── agents/           # Agno Agents
│   ├── keeper_agent.py
│   └── tools/        # Agent 工具
├── tests/
└── requirements.txt
```

系统架构图





技术栈关键特性

shadcn/ui 优势:

- 🎨 基于 Radix UI 的无障碍组件
- 📋 复制粘贴到项目, 完全可控
- 🎯 与 Tailwind CSS 完美集成
- 🛠️ TypeScript 原生支持
- 🌑 内置暗色模式

Agno 框架优势:

- 🤖 内置 LLM 抽象层 (支持 OpenAI/Claude/本地模型)
- 🧠 向量存储与检索 (Memory 组件)
- 🛠️ 工具调用框架 (Tool Calling)
- 📄 结构化输出 (Pydantic 验证)
- 🧑‍💻 Agent 编排与状态管理
- 🌐 丰富的集成 (LangChain 兼容)

4.4 MVP 定义

MVP = M0 + M1 (单人Web版)

后端功能:

- ✅ 规范冻结
- ✅ 单人单场景可玩 (检定/战斗/追逐)
- ✅ 基础规则问答
- ✅ 基础事件日志
- ✅ TRPG-only 门禁

前端界面:

- ✅ 用户注册/登录
- ✅ 角色卡创建/编辑界面
- ✅ 单人跑团游戏台 (聊天式交互)

- ☒ 状态面板 (HP/SAN/Leads 实时显示)
- ☒ 基础响应式布局 (桌面 + 平板)

5. 功能需求

详细规格说明： 每个里程碑的详细技术规格、数据结构定义和验收标准，请参阅 [详细规格文档](#)。

本文档按用户价值组织功能需求，详细的技术实施细节请参考详细规格文档。

5.1 核心游戏机制

FR-01: 自然语言交互系统

用户故事： 作为玩家，我希望用自然语言描述我的行动，而不需要记忆复杂的命令。

功能描述：

- 玩家输入默认被解释为"跑团内发言/行动/提问"
- 系统自动从语义中提取意图 (行动/提问/机制触发)
- 关键机制变更必须显式呈现 ([State] 块)
- TRPG-only 门禁：非跑团请求统一拒绝模板

验收标准：

玩家用自然语言描述行动，系统正确识别并推进
越界输入返回稳定拒绝模板
所有数值变更可追溯

FR-02: 检定系统

用户故事： 作为玩家，当我需要做技能检定时，系统应该自动处理掷骰、奖惩骰、推骰和花幸运。

功能描述：

- 支持属性检定 (STR/CON/DEX 等) 和技能检定
- 奖励骰/惩罚骰 (互斥)
- 推骰机制 (可推检定 + 更坏后果)
- 花幸运 (必须引用最近一次事件)
- 大成功/大失败判定

验收标准：

基础检定：目标值、掷骰、成功等级正确
奖惩骰互斥规则生效
推骰失败触发后果
花幸运正确追溯并修改结果

FR-03：战斗系统

用户故事： 作为玩家，当进入战斗时，系统应该自动管理回合、对抗、伤害结算，我只需要说"我攻击他"。

功能描述：

- 战斗态显式标记
- 回合与当前行动者清晰可见
- 近战对抗（双方检定，成功等级对比）
- 伤害结算（可审计）
- 重伤/濒死状态
- 治疗（引用受伤事件）

验收标准：

战斗流程可跑 3+ 回合
伤害落地可审计
濒死/重伤状态正确处理

FR-04：SAN 与疯狂系统

用户故事： 作为玩家，当我遭遇恐怖场景时，系统应该自动处理 SAN 检定、疯狂症状和恢复。

功能描述：

- SAN 检定（ /san check ）
- 临时疯狂/不定疯狂
- 疯狂症状（恐惧/狂躁/幻觉等）
- 疯狂恢复（治疗/时间）

验收标准：

SAN 检定触发疯狂状态
疯狂状态影响后续行动
恢复机制可追溯

FR-05：追逐系统

用户故事： 作为玩家，当需要逃跑或追赶时，系统应该管理距离、障碍和后果。

功能描述：

- 追逐态显式标记
- 距离/压力变量
- 障碍生成与应对
- 失败前进（新局面）

验收标准：

追逐流程可跑 5+ 回合
失败触发新局面（非原地踏步）

5.2 多人与协作

FR-06: 多人会话管理

用户故事: 作为 KP, 我希望邀请 2-4 名玩家加入跑团, 每个人都能看到自己该看的内容。

功能描述:

- 加入/离开/恢复
- 身份与角色绑定
- 聚光灯/队列系统
- 并发输入处理

验收标准:

2-4 人同团可稳定跑 10+ 轮
并发输入不冲突状态
掉线后可恢复

FR-07: 可见性控制

用户故事: 作为玩家, 我不应该看到其他玩家的私密线索或 KP 的秘密信息。

功能描述:

- 三档可见性 (公开/KP-only/私密)
- 所有输出按视角过滤
- 防止泄露 (history/recap/status)

验收标准:

KP-only 内容不泄露
私密线索正确隔离
检索/复盘遵循可见性

5.3 记忆与检索

FR-08: 事件日志

用户故事: 作为玩家, 我希望能够回顾过去发生了什么, 为什么我的 HP/SAN 会变化。

功能描述:

- Append-only 事件日志
- 最小字段 (event_id/timestamp/actor/type/payload/visibility)
- 状态变更可追溯到事件

验收标准:

任意数值变更可追溯到 event_id
日志不丢失不篡改
`/Log show` 可查看详情

FR-09：结构化摘要

用户故事： 作为玩家，当我中断一周后回来，我希望快速回忆起当前状态和关键线索。

功能描述：

- 检查点摘要（KP 显式创建）
- 场景摘要
- Session 总结
- 按视角过滤

验收标准：

`/recap` 输出稳定结构
恢复后状态与摘要一致
关键数值快照准确

FR-10：记忆检索

用户故事： 作为玩家，当我忘记"我们答应过谁什么"时，我希望能够搜索历史记录。

功能描述：

- 统一检索（ `/memory` ）
- 结果可引用
- 按视角过滤
- 防幻觉（优先引用证据）

验收标准：

检索结果包含引用 ID
KP 可见全部，玩家可见其权限内容
无证据时明确说明

5.4 内容与规则

FR-11：模组管理

用户故事： 作为 KP，我希望上传我的模组，系统自动校验格式并可用于开团。

功能描述：

- 模组上传（场景包格式）
- 结构校验（冻结字段）
- 版本管理
- 场景引用

验收标准：

有效模组可导入并校验通过
无效模组显示错误原因
模组要素可追溯引用

FR-12：规则知识库

用户故事：作为玩家，当我忘记某个规则时，我希望能够查询并获得带引用的答案。

功能描述：

- 规则书入库与分块
- 检索与引用 (citation_id)
- 证据优先级 (桌规 > 规则书 > 模组 > 裁量)
- 防注入 (资料即数据)

验收标准：

规则查询返回引用
/kb show 可展开上下文
注入文本不执行

FR-13：角色卡管理

用户故事：作为玩家，我希望创建和维护我的角色卡，并且可以在不同 Campaign 中使用。

功能描述：

- CRUD (创建/查看/编辑/归档)
- 导入/导出 (JSON)
- 字段遵循 M0-C 标准
- 分配给玩家

验收标准：

角色卡字段完整
导入导出一致
分配后权限正确

5.5 Web 界面

本产品为纯 Web 应用，以下界面是用户与系统交互的唯一方式。

FR-14：认证与权限界面

用户故事：作为用户，我希望能通过 Web 界面注册登录，并管理我的账号。

界面清单：

界面	功能	优先级
注册页	• 邮箱/用户名注册 • 密码强度提示 • 邮箱验证（可选）	P0
登录页	• 邮箱/用户名 + 密码登录 • 记住我 • 忘记密码	P0
个人中心	• 修改密码 • 个人信息 • 我的 Campaign 列表 • 我的角色卡库	P0

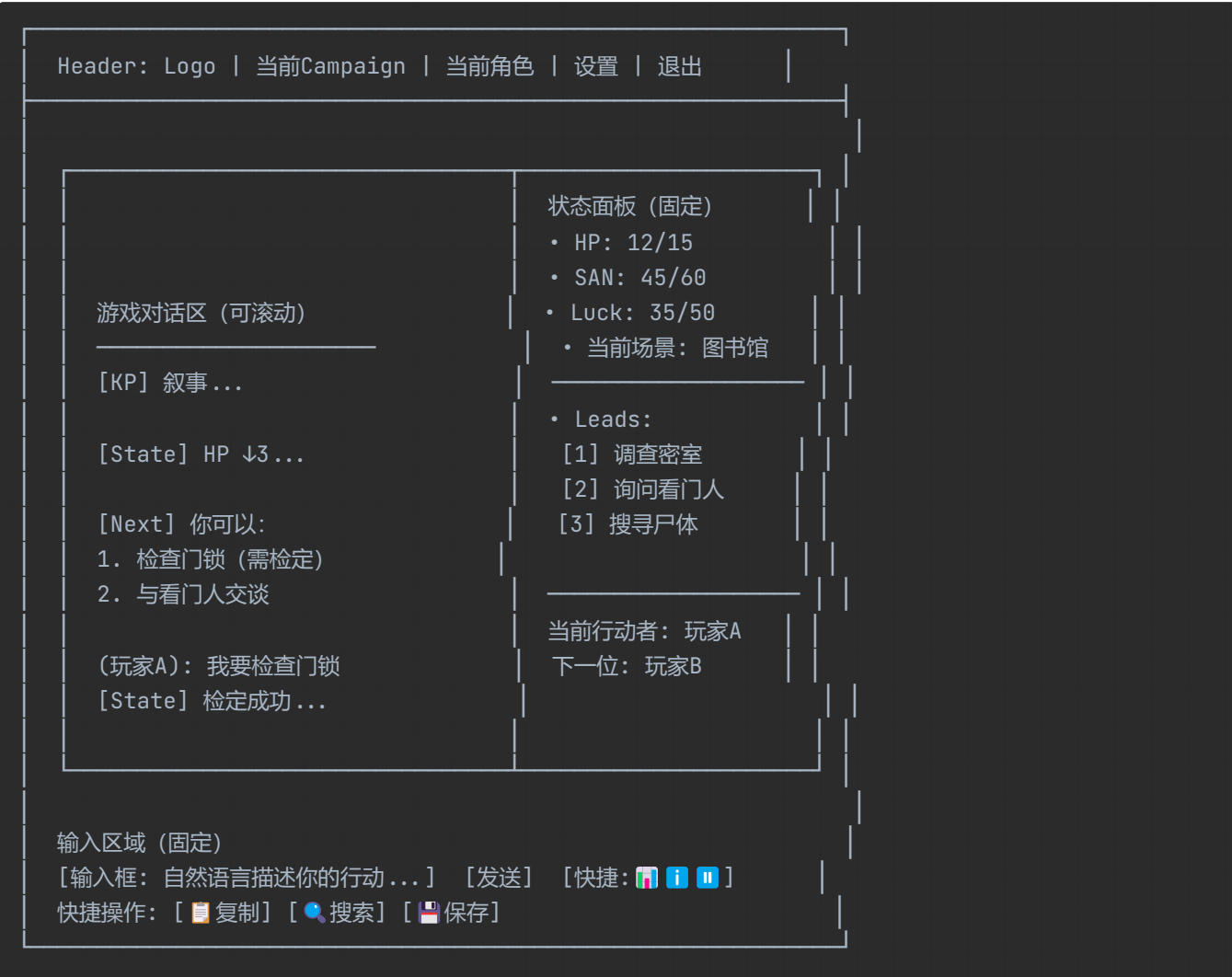
验收标准：

- 未登录访问受保护页面自动跳转登录
- 登录后正确识别用户身份（KP/Player）
- 权限控制生效（KP-only 内容对 Player 不可见）

FR-15：游戏台（核心界面）

用户故事： 作为玩家，我希望在浏览器中有一个沉浸式的跑团界面，能够输入自然语言、查看状态、回顾剧情。

界面布局：



核心组件：

组件	功能	优先级
对话区	- 消息气泡展示 - 区分 KP/玩家/OOC - 机制结果可视化（骰子/伤害）	P0
状态面板	- 实时 HP/SAN/Luck - 当前 Leads - 场景信息 - 计时器	P0
输入框	- 自然语言输入 - 自动完成建议 - 快捷命令按钮	P0
聚光灯指示	- 当前行动者高亮 - 队列预览 - 插入提示	P0（多人）

交互设计：

- 实时滚动新消息
- 机制结果动画效果（骰子滚动、数字跳动）
- 快捷操作无需输入命令
- 响应式布局（支持桌面/平板）

验收标准：

- 完成单人对战 5+ 轮，状态实时更新
- 多人场景下聚光灯指示正确
- KP-only 内容对 PLayer 不可见
- 移动端/平板可用性测试通过

FR-16：资源管理界面

用户故事：作为 KP，我希望通过 Web 界面上传模组、管理角色卡、创建 Campaign。

界面清单：

界面	功能	优先级
Campaign 列表	- 我的 Campaign - 创建新 Campaign - 加入 Campaign (通过邀请码)	P0
Campaign 详情	- 成员列表与管理 - 绑定脚本 - 分配角色卡 - Session 历史	P0
脚本库	- 脚本列表 (卡片式) - 上传脚本 (拖拽) - 脚本详情与校验结果 - 版本切换	P0
角色卡库	- 角色卡列表 - 创建/编辑角色卡 (表单式) - 导入/导出 JSON - 预览模式	P0
模组浏览器 (未来)	- 公开模组市场 - 评分与评论 - 一键导入	P1

验收标准：

- KP 可创建 Campaign 并邀请玩家
- 脚本上传后显示校验结果
- 角色卡编辑器所有字段可编辑
- 导入导出 JSON 格式正确

FR-17：复盘与检索界面

用户故事：作为玩家，当我中断后回来，我希望通过 Web 界面快速回顾剧情和搜索历史。

界面清单：

界面	功能	优先级
Session 列表	- 时间线展示 - Session 状态 (进行中/已结束) - 继续按钮	P0
复盘界面	- 结构化摘要展示 - 关键事件时间线 - 线索账本 - 角色卡快照	P0
搜索界面	- 统一搜索框 - 结果列表 (记忆/规则/模组) - 详情展开	P0
事件查看器	- 事件日志列表 - 事件详情 (按权限过滤) - 导出功能	P1

验收标准：

- 搜索结果按相关度排序
- KP 可查看完整事件日志
- 玩家只能查看其可见内容
- 复盘摘要与当前状态一致

FR-18：响应式设计

用户故事：作为玩家，我希望在电脑或平板上都能顺畅使用。

支持设备：

设备	支持程度	关键适配
桌面	完整支持	全功能可用
平板	完整支持	触控优化、布局适配
手机	基础支持	只读模式（查看剧情/状态），编辑功能降级

验收标准：

- 桌面/平板完整功能可用
- 手机可查看剧情和状态
- 横屏/竖屏自动适配
- 触控操作流畅

6. 非功能需求

6.1 性能要求

指标	目标	测量方法
响应时间	P95 < 3s (AI 生成)	监控日志
并发用户	支持 1000+ 并发 Session	压力测试
可用性	99.5% 月可用性	监控告警
数据持久	事件日志零丢失	数据库备份与验证

6.2 安全要求

要求	具体措施
认证与授权	JWT Token, RBAC (KP/Player)
数据隔离	Campaign 级数据隔离, KP-only 内容加密存储
输入防护	提示注入检测, 知识库资料防注入
审计日志	敏感操作（登录/权限变更/Retcon）留痕
隐私保护	密码哈希, 个人信息脱敏

6.3 可维护性要求

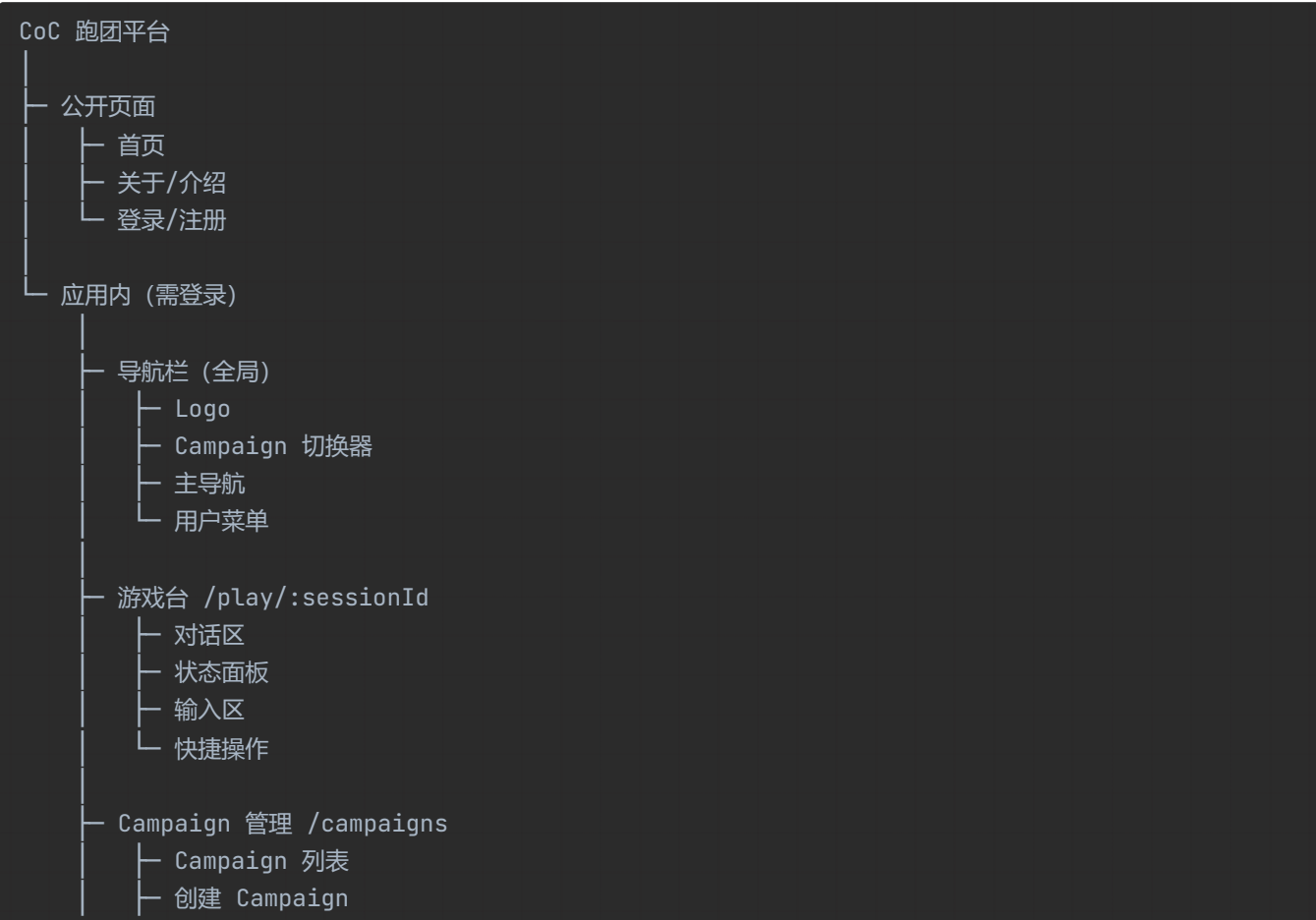
要求	具体措施
可扩展性	模块化设计，支持新增规则系统/语言
可观测性	结构化日志，指标导出，链路追踪
可测试性	机制逻辑可单元测试，E2E 测试覆盖核心流程
文档	API 文档，用户手册，开发者指南

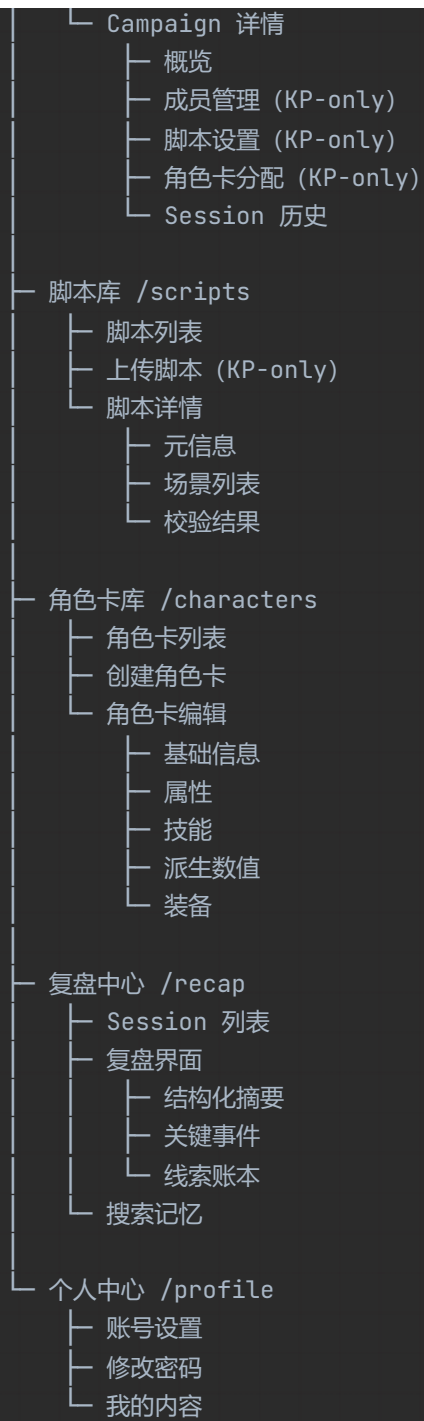
6.4 AI 特定要求

要求	具体措施
门禁强度	TRPG-only 门禁，越界拒绝率 > 99%
机制准确性	检定/伤害/SAN 计算零错误（双层验证：LLM + 规则引擎）
幻觉控制	知识库检索优先，无证据时明确说明
输出稳定性	结构化块（[KP]/[State]/[Next]）解析成功率 > 99%

7. Web 应用架构

7.1 信息架构 (IA)





7.2 页面优先级

页面	优先级	所属里程碑	说明
登录/注册页	P0	M1	用户入口
角色卡创建/编辑	P0	M1	必需功能
游戏台（单人）	P0	M1	核心界面
Campaign 列表/详情	P0	M1	组织单元
游戏台（多人）	P0	M2	多人支持
脚本库	P0	M4	资源管理
复盘中心	P0	M3	长记忆
搜索界面	P1	M3	记忆检索
首页/介绍页	P1	M6	营销落地

7.3 路由设计 (React Router v6 + shadcn/ui)

```
// App.tsx - 应用路由结构
import { createBrowserRouter } from 'react-router-dom'
import { ProtectedRoute } from '@components/auth/ProtectedRoute'
import { AppLayout } from '@components/layout/AppLayout'

export const router = createBrowserRouter([
  // 公开页面
  {
    path: '/',
    element: <HomePage />,
  },
  {
    path: '/login',
    element: <LoginPage />,
  },
  {
    path: '/register',
    element: <RegisterPage />,
  },
  // 受保护页面（需登录）
  {
    path: '/app',
    element: <ProtectedRoute><AppLayout /></ProtectedRoute>,
    children: [
      // 默认重定向到 Campaign 列表
      { index: true, redirect: 'campaigns' },

      // 游戏台（核心界面）
      {
        path: 'play/:sessionId',
```

```
    element: <GameConsole />,
    // 使用 shadcn/ui 的 Card 组件作为容器
  },

  // Campaign 管理
  {
    path: 'campaigns',
    element: <CampaignList />,
    // 使用 shadcn/ui 的 DataTable 组件
  },
  {
    path: 'campaigns/new',
    element: <CreateCampaign />,
    // 使用 shadcn/ui 的 Form 组件
  },
  {
    path: 'campaigns/:id',
    element: <CampaignDetail />,
    // 使用 shadcn/ui 的 Tabs 组件
  },

  // 脚本库
  {
    path: 'scripts',
    element: <ScriptLibrary />,
    // 使用 shadcn/ui 的 Card + Grid 组件
  },
  {
    path: 'scripts/upload',
    element: <UploadScript />,
    // KP-only 路由, 使用 ProtectedRoute 检查权限
    // 使用 shadcn/ui 的 Upload 组件
  },
  {
    path: 'scripts/:id',
    element: <ScriptDetail />,
    // 使用 shadcn/ui 的 Accordion 组件
  },

  // 角色卡库
  {
    path: 'characters',
    element: <CharacterLibrary />,
    // 使用 shadcn/ui 的 Card 组件
  },
  {
    path: 'characters/new',
    element: <CreateCharacter />,
    // 使用 shadcn/ui 的 Form + Tabs 组件
  },
  {
    path: 'characters/:id/edit',
```

```

        element: <EditCharacter />,
        // 使用 shadcn/ui 的 Form 组件
    },

    // 复盘中心
    {
        path: 'recap',
        element: <RecapCenter />,
        // 使用 shadcn/ui 的 Timeline 组件
    },
    {
        path: 'recap/:sessionId',
        element: <SessionRecap />,
        // 使用 shadcn/ui 的 ScrollArea + Badge 组件
    },
    {
        path: 'search',
        element: <SearchPage />,
        // 使用 shadcn/ui 的 Command + Input 组件
    },

    // 个人中心
    {
        path: 'profile',
        element: <Profile />,
        // 使用 shadcn/ui 的 Card + Avatar 组件
    },
],
},

// 404 页面
{
    path: '*',
    element: <NotFoundPage />,
},
])

// ProtectedRoute 组件示例
export const ProtectedRoute = ({ children }: { children: React.ReactNode }) => {
    const { user, loading } = useAuth()
    const location = useLocation()

    if (loading) {
        return <LoadingPage /> // 使用 shadcn/ui 的 Spinner 组件
    }

    if (!user) {
        return <Navigate to="/login" state={{ from: location }} replace />
    }

    return <>{children}</>
}

```

```
// KP-only 路由守卫示例
export const KPRoute = ({ children }: { children: React.ReactNode }) => {
  const { user } = useAuth()

  if (user?.role !== 'kp') {
    return <Navigate to="/app/campaigns" replace />
  }

  return <>{children}</>
}
```

shadcn/ui 组件使用示例:

```
// 游戏台界面组件示例
import { Card } from '@components/ui/card'
import { Button } from '@components/ui/button'
import { Input } from '@components/ui/input'
import { Badge } from '@components/ui/badge'
import { ScrollArea } from '@components/ui/scroll-area'
import { Separator } from '@components/ui/separator'

export function GameConsole() {
  return (
    <div className="flex h-screen">
      {/* 对话区 */}
      <div className="flex-1 p-4">
        <ScrollArea className="h-full">
          {/* 消息列表 */}
        </ScrollArea>
      </div>

      <Separator orientation="vertical" />

      {/* 状态面板 */}
      <Card className="w-80 p-4">
        <h3 className="font-semibold">状态</h3>
        <Badge variant="outline">HP: 12/15</Badge>
        <Badge variant="destructive">SAN: 45/60</Badge>
      </Card>
    </div>
  )
}
```

7.4 权限控制

页面/功能	游客	Player	KP
首页/介绍	✓	✓	✓
登录/注册	✓	✗	✗
游戏台 (玩家)	✗	✓	✓
创建 Campaign	✗	✓	✓
Campaign 管理成员	✗	✗	✓
上传脚本	✗	✗	✓
编辑他人角色卡	✗	✗	✓
KP-only 内容查看	✗	✗	✓

8. 用户体验设计原则

7.1 核心原则

- 1. 自然语言优先
 - 玩家不需要学习命令，自然语言即可
 - 命令作为快捷方式，而非必需
- 2. 机制显式可见
 - 所有数值变更必须显式呈现
 - 玩家始终知道"为什么变了"
- 3. TRPG-only 门禁友好
 - 拒绝模板：原因 + 允许的替代 + 下一步建议
 - 不说"不"，说"试试这样"
- 4. 防卡死设计
 - Leads 机制：始终提供 2-4 个可行动方向
 - 失败前进：失败不是卡住，而是代价不同
- 5. 按视角过滤
 - 玩家只看到他们该看到的
 - KP 享有全局视角

7.2 输出格式标准

标准输出结构（每轮）

```
[KP]
叙事与裁定 (1-3 段, 根据 format 设置调整长度)

[State]
• HP: 12/15 (↓3 from 战斗伤害 #142)
• SAN: 45/60 (↓5 from 目击尸体 #138)
• 当前场景: 图书馆 - 夜间
• 计时器: 警方将在 2 小时后介入
• Leads: [1] 调查密室 [2] 询问看门人 [3] 搜寻尸体

[Next]
你可以:
1. 检查密室的门锁 (需要检定)
2. 与看门人交谈 (自然语言)
3. 在尸体旁搜寻线索 (需要检定)
```

拒绝模板

```
[Refusal]
这个请求超出了跑团范围。我无法帮你写论文。

[Allowed]
在跑团中, 你可以:
• /act "我要在图书馆查阅资料"
• /rule "怎么进行图书馆使用检定?"
• /help 查看更多命令

[Next]
试试说: "我想在图书馆查找关于这个线索的资料"
```

9. 数据模型

9.1 核心实体关系

```
Account (用户)
├─ Campaign (跑团/团) [1:N]
│   ├── Session (会话) [1:N]
│   │   └─ Event (事件) [1:N]
│   ├── Member (成员) [N:M]
│   ├── Script (绑定脚本) [N:1]
│   └─ Character (角色卡) [1:N]
├─ Script (脚本库) [1:N]
└─ Character (角色卡库) [1:N]
```

```
Script (脚本)
└─ Scene (场景) [1:N]
    ├── NPC [1:N]
    ├── Location [1:N]
    ├── Clue (线索) [1:N]
    └─ Handout (手递物) [1:N]

KnowledgeBase (知识库)
└─ Citation (引用片段) [1:N]
```

9.2 关键数据结构

Account (用户)

```
interface Account {
  account_id: string
  username: string
  email: string
  password_hash: string
  created_at: timestamp
  roles: Role[] // 全局角色 (如管理员)
}
```

Campaign (跑团)

```
interface Campaign {
  campaign_id: string
  name: string
  owner_account_id: string // KP
  script_id: string | null
  script_version: string | null
  status: 'active' | 'archived'
  members: CampaignMember[]
  created_at: timestamp
}
```

Session (会话)

```
interface Session {
  session_id: string
  campaign_id: string
  current_scene_id: string | null
  state: SessionState // 包含 HP/SAN/Leads 等
  status: 'active' | 'paused' | 'ended'
  started_at: timestamp
  ended_at: timestamp | null
}
```

Event（事件）

```
interface Event {
  event_id: string
  session_id: string
  timestamp: timestamp
  actor_player_id: string | null
  actor_role: 'KP' | 'Player'
  controlled_character_id: string | null
  type: EventType
  payload: object
  visibility: 'public' | 'kp' | 'player:<id>'
}
```

Character（角色卡）

```
interface Character {
  character_id: string
  account_id: string | null // owner
  campaign_id: string | null
  name: string
  // 基础字段（MO-C 冻结）
  attributes: {
    STR: number; CON: number; DEX: number; APP: number;
    POW: number; INT: number; SIZ: number; EDU: number;
  }
  derived: {
    HP: number; HP_max: number;
    MP: number; MP_max: number;
    SAN: number; SAN_max: number;
    Luck: number; Luck_max: number;
    Move: number;
  }
  skills: Record<string, number>
  status: CharacterStatus
  inventory: string[]
  // ...
}
```

10. 里程碑规划

10.1 里程碑概览

重要说明：本产品为 Web 应用，每个里程碑都包含对应的前端界面开发。Web 界面不是“附加功能”，而是产品的核心交互方式。

里程碑	目标	后端核心功能	前端界面	预估周期
M0	规范冻结	命令集/场景包/状态字段定义	UI 设计规范/组件库规划	1 周
M1	单人Web版	检定/战斗/追逐/规则问答闭环	登录/注册/单人跑团界面/基础游戏台	4 周
M2	多人Web版	并发/可见性/权限/聚光灯	多人席位显示/实时聊天/聚光灯UI	3 周
M3	记忆Web版	事件日志/摘要/检索/恢复	复盘界面/记忆搜索/时间线展示	3 周
M4	资源管理Web版	模组导入/规则书检索/统一搜索	脚本库/角色卡库/资源上传管理界面	2 周
M5	全功能Web版	SAN/疯狂/完整战斗/成长/短模组验收	完整游戏UI/状态面板/机制可视化	4 周
M6	体验打磨	防卡死/Leads/友好拒绝/输出优化	交互优化/响应式适配/加载体验	2 周

总计：约 19 周 (约 5 个月)

10.2 里程碑依赖关系



10.3 MVP 定义

MVP = M0 + M1 (包含Web界面)

后端功能：

- ✔ 规范冻结
- ✔ 单人单场景可玩 (检定/战斗/追逐)
- ✔ 基础规则问答
- ✔ 基础事件日志
- ✔ TRPG-only 门禁

前端界面：

- ✔ 用户注册/登录
- ✔ 角色卡创建/编辑界面

- ☒ 单人跑团游戏台（聊天式交互）
- ☒ 状态面板（HP/SAN/Leads实时显示）
- ☒ 基础响应式布局

MVP 后续迭代：

迭代	后端目标	前端增值
MVP+1	多人支持	多人席位/实时聊天/聚光灯指示
MVP+2	长记忆	复盘界面/搜索框/时间线组件
MVP+3	知识库	资源库界面/模组浏览/规则检索
MVP+4	全功能	完整战斗UI/SAN可视化/状态动画
MVP+5	体验打磨	交互优化/移动端适配/性能优化

11. 成功指标

11.1 产品指标（上线后 6 个月）

指标	目标	测量方法
注册用户	5,000+	数据库统计
活跃用户 (MAU)	1,500+	用户登录记录
创建 Campaign 数	500+	Campaign 表统计
完成 Session 数	2,000+	Event 日志统计
平均 Session 时长	90+ 分钟	时间戳差值
用户留存 (D7)	30%+	注册后 7 日回访率
用户留存 (D30)	15%+	注册后 30 日回访率

11.2 质量指标

指标	目标	测量方法
机制错误率	< 0.1%	人工抽检 + 用户反馈
门禁误拒率	< 1%	拒绝日志人工复核
系统可用性	> 99.5%	监控系统
响应时间 P95	< 3s	APM 监控
用户满意度	> 4.0/5.0	问卷调研

11.3 业务指标

指标	目标	测量方法
付费转化率	5%+	付费用户 / 活跃用户
ARPU	\$10+	收入 / 活跃用户
CAC	< \$20	市场投入 / 新增用户
LTV/CAC	> 3	用户生命周期价值 / 获客成本

附录

A. 术语表

术语	英文	定义
KP	Keeper	主持人/游戏管理员 (GM)
Player	-	玩家
PC	Player Character	玩家角色
NPC	Non-Player Character	非玩家角色
Session	-	一次跑团会话 (可跨天/跨周继续)
Scene	-	场景 (模组的一级叙事单元)
Campaign	-	跑团/团 (长期组织单元)
CoC 7e	Call of Cthulhu 7th Edition	克苏鲁的呼唤第 7 版规则
SAN	Sanity	理智值
TRPG	Tabletop Role-Playing Game	桌面角色扮演游戏

B. 参考文档

- 详细规格文档: docs/prd/detailed-specs.md (里程碑 M0-M6 详细技术规格)
- CoC 7e 规则书
- 里程碑映射: 见本文档第 10 节 (里程碑规划)

C. 技术栈详细说明

C.1 前端依赖 (React + shadcn/ui)

核心依赖:

```
{
  "dependencies": {
    "react": "^18.3.0",
    "react-dom": "^18.3.0",
```

```
    "react-router-dom": "^6.22.0",
    "zustand": "^4.5.0",
    "socket.io-client": "^4.7.0",
    "@tanstack/react-query": "^5.28.0",
    "react-hook-form": "^7.51.0",
    "zod": "^3.22.0",
    "@hookform/resolvers": "^3.3.0",
    "tiptap": "^2.3.0",
    "date-fns": "^3.6.0"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.2.0",
    "vite": "^5.2.0",
    "typescript": "^5.4.0",
    "tailwindcss": "^3.4.0",
    "@tailwindcss/typography": "^0.5.0",
    "autoprefixer": "^10.4.0",
    "postcss": "^8.4.0",
    "eslint": "^8.57.0",
    "prettier": "^3.2.0",
    "@types/react": "^18.2.0"
  }
}
```

shadcn/ui 组件清单 (按功能分组)：

功能模块	使用组件	说明
基础组件	Button, Input, Textarea, Label	表单输入
布局	Card, Separator, ScrollArea, Tabs	页面布局
数据展示	Badge, Avatar, Table, DataTable	信息展示
反馈	Alert, Toast, Dialog, Alert	用户反馈
导航	Navigation Menu, Breadcrumb, Pagination	页面导航
表单	Form, Select, Checkbox, Radio, Switch	复杂表单
游戏特定	Progress (HP/SAN条), Tooltip (帮助提示), Popover	游戏UI
其他	Command (搜索快捷键), HoverCard (角色卡预览)	交互增强

C.2 后端依赖 (Python + Agno)

核心依赖：

```
# Web 框架
fastapi ≥ 0.110.0
uvicorn[standard] ≥ 0.29.0
python-multipart ≥ 0.0.9
python-jose[cryptography] ≥ 3.3.0
passlib[bcrypt] ≥ 1.7.4
python-dotenv ≥ 1.0.1
```

```
# Agno 框架
agno-ai ≥ 0.1.0

# 数据库
sqlalchemy ≥ 2.0.29
alembic ≥ 1.13.0
asyncpg ≥ 0.29.0
redis ≥ 5.0.3

# AI/ML
openai ≥ 1.14.0
anthropic ≥ 0.25.0
langchain ≥ 0.1.0
langchain-openai ≥ 0.1.0

# 向量检索
pgvector ≥ 0.2.5
# 或
pinecone-client ≥ 3.0.0

# 任务队列
celery ≥ 5.3.6
flower ≥ 2.0.1

# WebSocket
socketio ≥ 5.11.0
python-socketio ≥ 5.11.0
aiohttp ≥ 3.9.5

# 工具库
pydantic ≥ 2.6.0
pydantic-settings ≥ 2.2.0
httpx ≥ 0.27.0
loguru ≥ 0.7.2

# 开发工具
pytest ≥ 8.1.0
pytest-asyncio ≥ 0.23.6
pytest-cov ≥ 5.0.0
black ≥ 24.3.0
ruff ≥ 0.3.0
mypy ≥ 1.9.0
```

Agno Agent 组件:

```
# Keeper Agent 示例结构
from agno import Agent, Toolkit
from agno.llm.openai import OpenAILLM

# 游戏机制工具包
from app.agents.tools.game_tools import (
```



```

    RollTool,      # 掷骰工具
    DamageTool,    # 伤害计算工具
    SANToll,       # SAN 检定工具
    CombatTool,    # 战斗工具
)

# 记忆工具包
from app.agents.tools.memory_tools import (
    RecallTool,    # 记忆检索
    StoreTool,     # 记忆存储
)

# 知识库工具包
from app.agents.tools.kb_tools import (
    SearchRulesTool, # 搜索规则
    SearchModuleTool, # 搜索模组
)

# 创建 Keeper Agent
keeper_agent = Agent(
    name="keeper",
    role="CoC Keeper (Game Master)",
    goal="主持 CoC 跑团, 推动剧情, 裁定规则",
    backstory="你是一个经验丰富的 CoC Keeper ...",
    llm=OpenAILLM(model="gpt-4-turbo"),
    tools=[
        RollTool(),
        DamageTool(),
        SANToll(),
        CombatTool(),
        RecallTool(),
        StoreTool(),
        SearchRulesTool(),
        SearchModuleTool(),
    ],
    # 结构化输出配置
    response_format=KeeperOutput,
    # 门禁控制
    guardrails=TRPGGuardrail(),
)

```

C.3 开发工具链

前端开发:

```

# 初始化项目
npm create vite@latest coc-frontend -- --template react-ts
cd coc-frontend

# 安装 shadcn/ui
npx shadcn-ui@latest init

```

```
# 添加常用组件
npx shadcn-ui@latest add button card input textarea
npx shadcn-ui@latest add badge avatar scroll-area
npx shadcn-ui@latest add dialog toast alert
npx shadcn-ui@latest add form select checkbox

# 开发
npm run dev

# 构建
npm run build

# 类型检查
npm run type-check

# 代码检查
npm run lint
```

后端开发:

```
# 创建虚拟环境
python -m venv venv
source venv/bin/activate # Linux/Mac
# venv\Scripts\activate # Windows

# 安装依赖
pip install -r requirements.txt

# 数据库迁移
alembic upgrade head

# 运行开发服务器
uvicorn app.main:app --reload --port 8000

# 运行测试
pytest

# 代码格式化
black app/
ruff check app/
mypy app/
```

C.4 部署配置

前端部署 (Vercel 示例) :

```
# 安装 Vercel CLI
npm i -g vercel

# 部署
vercel --prod
```

后端部署 (Docker) :

```
# Dockerfile
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# docker-compose.yml
version: '3.8'
services:
  backend:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://...
      - REDIS_URL=redis://redis:6379
    depends_on:
      - postgres
      - redis

  postgres:
    image: postgres:16
    environment:
      - POSTGRES_DB=coc_db
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=pass
    volumes:
      - postgres_data:/var/lib/postgresql/data

  redis:
    image: redis:7
    volumes:
      - redis_data:/data

volumes:
  postgres_data:
  redis_data:
```

D. 变更记录

日期	版本	变更内容	变更人
2026-02-05	v1.0	初始版本, 从原始需求整理	-
2026-02-05	v1.1	- 明确产品为纯 Web 应用 - 更新里程碑规划 (M1-M6, Web 融入各阶段) - 新增"Web 应用架构"章节 - 技术栈确定: React + shadcn/ui / Python + Agno	-
2026-02-05	v1.2	- 创建详细规格文档 (detailed-specs.md) - 提取所有里程碑详细规格到独立文档 - 移除对原始需求文档的引用 - 所有技术细节统一在详细规格文档管理	-

文档状态:  已完成初稿, 待评审